BusyBox



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Why BusyBox?

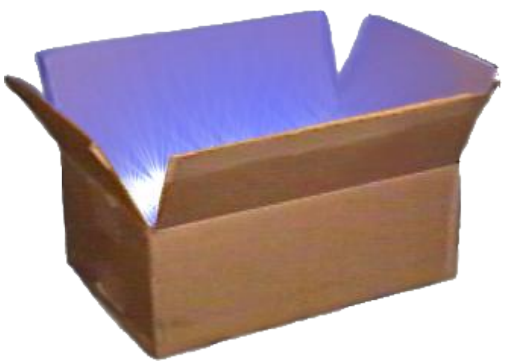
- A Linux system needs a basic set of programs to work
  - An init program
  - A shell
  - Various basic utilities for file manipulation and system configuration
- In normal GNU/Linux systems, these programs are provided by different projects
  - `coreutils`, `bash`, `grep`, `sed`, `tar`, `wget`, `modutils`, etc. are all different projects
  - A lot of different components to integrate
  - Components not designed with embedded systems constraints in mind: they are not very configurable and have a wide range of features
- BusyBox is an alternative solution, extremely common on embedded systems



General purpose toolbox: BusyBox

https://www.busybox.net/

- Rewrite of many useful UNIX command line utilities
  - Created in 1995 to implement a rescue and installer system for Debian, fitting in a single floppy disk (1.44 MB)
  - Integrated into a single project, which makes it easy to work with
  - Great for embedded systems: highly configurable, no unnecessary features
  - Called the *Swiss Army Knife of Embedded Linux*
- License: GNU GPLv2
- Alternative: Toybox, BSD licensed (https://en.wikipedia.org/wiki/Toybox)





BusyBox in the root filesystem

- All the utilities are compiled into a single executable, `/bin/busybox`
  - Symbolic links to `/bin/busybox` are created for each application integrated into BusyBox
- For a fairly featureful configuration, less than 500 KB (statically compiled with uClibc) or less than 1 MB (statically compiled with glibc).

```
rootfs
├── bin
│   ├── ash -> busybox
│   ├── busybox
│   ├── cat -> busybox
│   ├── ls -> busybox
│   ├── mount -> busybox
│   └── sh -> busybox
├── sbin
│   ├── halt -> ../bin/busybox
│   ├── ifconfig -> ../bin/busybox
│   └── init -> ../bin/busybox
└── usr
    └── sbin
        └── httpd -> ../../bin/busybox
```



BusyBox - Most commands in one binary

```
[, [[, acpid, addgroup, adduser, adjtimex, ar, arp, arping, ash, awk, basename, beep, blkid, brctl, bunzip2, bzcat, bzip2, cal, cat,
catv, chat, chattr, chgrp, chmod, chown, chpasswd, chpst, chroot, chrt, chvt, cksum, clear, cmp, comm, cp, cpio, crond, crontab,
cryptpw, cut, date, dc, dd, deallocvt, delgroup, deluser, depmod, devmem, df, dhcprelay, diff, dirname, dmesg, dnsd, dnsdomainname,
dos2unix, dpkg, du, dumpkmap, dumpleases, echo, ed, egrep, eject, env, envdir, envuidgid, expand, expr, fakeidentd, false, fbset,
fbsplash, fdflush, fdformat, fdisk, fgrep, find, findfs, flash_lock, flash_unlock, fold, free, freeramdisk, fsck, fsck.minix, fsync,
ftpd, ftpget, ftpput, fuser, getopt, getty, grep, gunzip, gzip, hd, hdparm, head, hexdump, hostid, hostname, httpd, hush, hwclock,
id, ifconfig, ifdown, ifenslave, ifplugd, ifup, inetd, init, inotifyd, insmod, install, ionice, ip, ipaddr, ipcalc, ipcrm, ipcs,
iplink, iproute, iprule, iptunnel, kbd_mode, kill, killall, killall5, klogd, last, length, less, linux32, linux64, linuxrc, ln,
loadfont, loadkmap, logger, login, logname, logread, losetup, lpd, lpq, lpr, ls, lsattr, lsmod, lzmacat, lzop, lzopcat, makemime,
man, md5sum, mdev, mesg, microcom, mkdir, mkdosfs, mkfifo, mkfs.minix, mkfs.vfat, mknod, mkpasswd, mkswap, mktemp, modprobe, more,
mount, mountpoint, mt, mv, nameif, nc, netstat, nice, nmeter, nohup, nslookup, od, openvt, passwd, patch, pgrep, pidof, ping, ping6,
pipe_progress, pivot_root, pkill, popmaildir, printenv, printf, ps, pscan, pwd, raidautorun, rdate, rdev, readlink, readprofile,
realpath, reformime, renice, reset, resize, rm, rmdir, rmmod, route, rpm, rpm2cpio, rtcwake, run-parts, runlevel, runsv, runsvdir,
rx, script, scriptreplay, sed, sendmail, seq, setarch, setconsole, setfont, setkeycodes, setlogcons, setsid, setuidgid, sh, sha1sum,
sha256sum, sha512sum, showkey, slattach, sleep, softlimit, sort, split, start-stop-daemon, stat, strings, stty, su, sulogin, sum,
sv, svlogd, swapoff, swapon, switch_root, sync, sysctl, syslogd, tac, tail, tar, taskset, tcpsvd, tee, telnet, telnetd, test, tftp,
tftpd, time, timeout, top, touch, tr, traceroute, true, tty, ttysize, udhcpc, udhcpd, udpsvd, umount, uname, uncompress, unexpand,
uniq, unix2dos, unlzma, unlzop, unzip, uptime, usleep, uudecode, uuencode, vconfig, vi, vlock, volname, watch, watchdog, wc, wget,
which, who, whoami, xargs, yes, zcat, zcip
```

Source: https://busybox.net/downloads/BusyBox.txt - July 2026 status



Configuring BusyBox

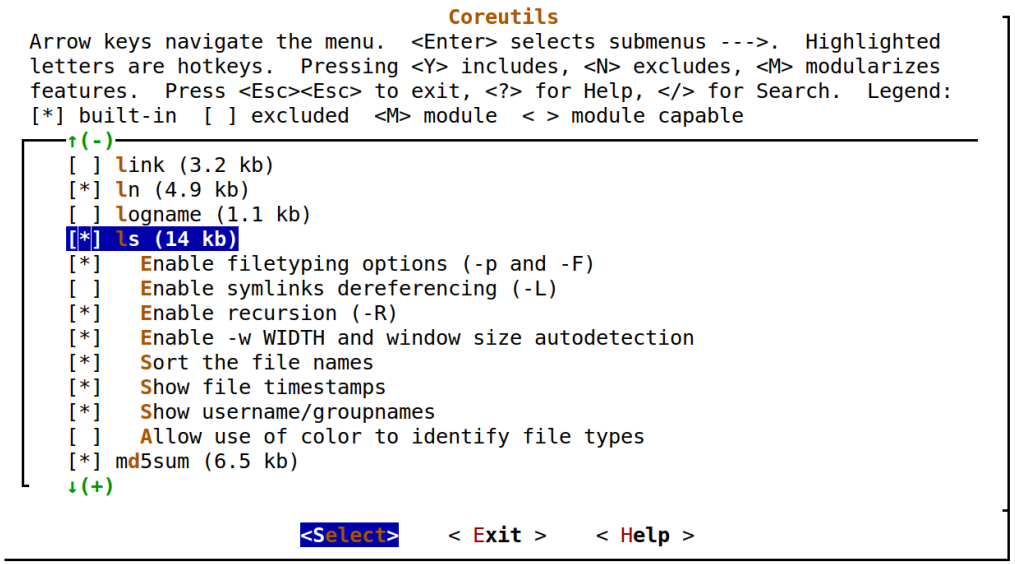
- Get the latest stable sources from https://busybox.net
- Configure BusyBox (creates a `.config` file):
  - `make defconfig`
    Good to begin with BusyBox. Configures BusyBox with all options for regular users.
  - `make allnoconfig`
    Unselects all options. Good to configure only what you need.
- `make menuconfig` (text)
  Same configuration interfaces as the ones used by the Linux kernel (though older versions are used, causing `make xconfig` to be broken in recent distros).



BusyBox make menuconfig

You can choose:

- the commands to compile,
- and even the command options and features that you need!





Compiling BusyBox

- Set the cross-compiler prefix:
  - from the menuconfig interface
    `Settings → Build Options → Cross Compiler prefix`
  - or through the `CROSS_COMPILE` environment variable
    `export CROSS_COMPILE=arm-linux-`
- Set the installation directory in the configuration interface:
  `Settings → Installation Options → Destination path for 'make install'`
- Add the cross-compiler path to the PATH environment variable:
  `export PATH=$HOME/x-tools/arm-unknown-linux-uclibcgnueabi/bin:$PATH`
- Compile BusyBox:
  `make`
- Install it (this creates a UNIX directory structure with symbolic links to the `busybox` executable):
  `make install`



Applet highlight: BusyBox init

- BusyBox provides an implementation of an `init` program
- Simpler than the init implementation found on desktop/server systems (*SysV init* or *systemd*)
- A single configuration file: `/etc/inittab`
  - Each line has the form `<id>::<action>:<process>`
- Allows to start system services at startup, to control system shutdown, and to make sure that certain services are always running on the system.
- See examples/inittab in BusyBox for details on the configuration

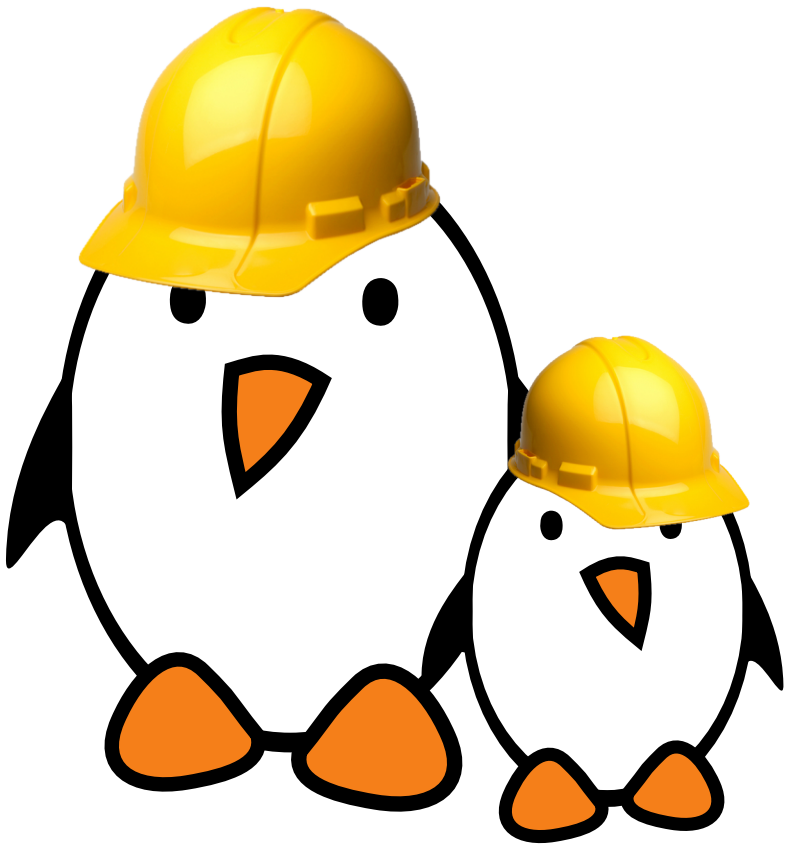


Applet highlight: BusyBox vi

- If you are using BusyBox, adding `vi` support only adds about 20K
- You can select which exact features to compile in.
- Users hardly realize that they are using a lightweight `vi` version!
- Tip: you can learn `vi` on the desktop, by running the `vimtutor` command.

```
[ ] cmp (4.9 kb)
[ ] diff (13 kb)
[ ] ed (21 kb)
[ ] patch (9.4 kb)
[ ] sed (12 kb)
[*] vi (23 kb)
(4096) Maximum screen width
[*]   Allow to display 8-bit chars (otherwise shows dots)
[*]   Enable ":" colon commands (no "ex" mode)
[*]   Enable yank/put commands and mark cmds
[*]   Enable search and replace cmds
[ ]     Enable regex in search and replace
[*]   Catch signals
[*]   Remember previous cmd and "." cmd
[*]   Enable -R option and "view" mode
[*]   Enable settable options, ai ic showmatch
[*]   Support :set
[ ]   Handle window resize
[ ]   Use 'tell me cursor position' ESC sequence to measure window
[*]   Support undo command "u"
[*]     Enable undo operation queuing
(256)     Maximum undo character queue size
[ ]   Allow vi and awk to execute shell commands
```

Practical lab - Tiny root filesystem built from scratch with BusyBox



- Setting up a kernel to boot your system on a workstation directory exported by NFS
- Passing kernel command line parameters to boot on NFS
- Creating the full root filesystem from scratch. Populating it with BusyBox based utilities.
- System startup using BusyBox `init`
- Using the BusyBox HTTP server.
- Controlling the target from a web browser on the PC host.
- Setting up shared libraries on the target and compiling a sample executable.